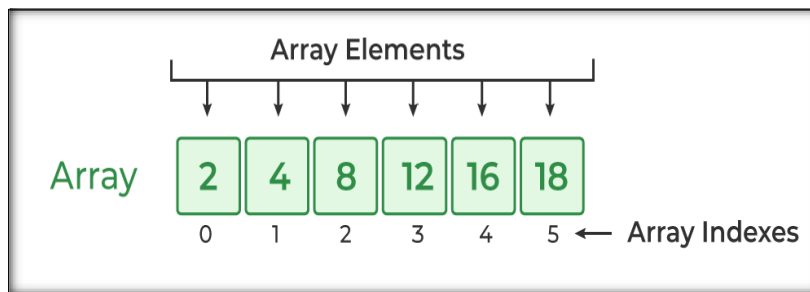


Linear - Static Data Structure (Arrays)

Static data structure has a **fixed memory size**. It is easier to access the elements in a static data structure. An example of this data structure is an Array.

An array is a linear data structure and it is a **collection** of items stored at contiguous memory locations (مواقع ذاكرة متجاورة). The idea is to store **multiple items** of the **same type** together in **one place**. It allows the processing of a large amount of data in a **relatively short period** (فترة قصيرة نسبياً). The **first element** of an array is **indexed** by a subscript of (0). We can perform several different operations on the elements of the array, including searching, deleting, sorting, inserting, Reversing, and replacing.



➤ Characteristics of an Array:

1. Arrays use an index-based data structure which helps to identify each of the elements in an array easily using the index.
2. If a user wants to store multiple values of the same data type, then the array can be utilized efficiently.
3. An array can also handle complex data structures by storing data in a two- dimensional array.
4. An array is also used to implement other data structures like Stacks, Queues, Heaps, Hash tables, etc.
5. The search process in an array can be done very easily.

➤ Basic Operations performed on array

These are some of the most common operations performed on arrays. The specific operations and algorithms used may vary based on the requirements of the problem and the programming language used:

1. Initialization: It means defining the **type of array** and **assigning it values**. The simplest way to initialize an array is by using static arrays, which are arrays with fixed size and values assigned when declared. Arrays can also be initialized dynamically during runtime, meaning that the size of the array and its elements are determined as the program is being executed.
2. Accessing elements: Elements in an array can be accessed by their index, which starts from 0 and goes up to the size of the array minus one.
3. Searching for elements: Looking for something specific? Searching is the answer. It's a process of identifying an element from within an array by comparing it to your desired value until you find a match. There are two distinct types of searches: **linear** and **binary** search techniques, both offering varying degrees of efficiency when used correctly. Linear search compares each element one after another until a match is found or all elements have been searched.
4. Sorting elements: Sorting is a process of arranging elements of an array in either ascending or descending order (تصاعدي او تنازلي). Array sorting can be done using different algorithms:
 - a. Bubble sort,
 - b. Insertion sort,
 - c. Selection sort, and
 - d. Quick sort.
5. Inserting elements: Elements can be inserted into an array at a specific location, but this operation can be time-consuming because it requires shifting existing (other elements) in the array to make space for the insertion.
6. Deleting elements: Deletion is the opposite of insertion and involves removing elements from an existing array. After deleting an element, all the elements that come after it must be shifted to fill any gaps left from deletion.
7. Updating elements: Elements in an array can be updated or modified by assigning a new value to a specific index.
8. Traversing : Traversing arrays involves looping through each element in the array and processing each element one at a time. This allows you to access all array elements and perform tasks such as printing, copying, comparing, or sorting.

➤ Applications of Array:

Different applications of an array are as follows:

1. Database records are also implemented by an array.
2. It helps in implementing a sorting algorithm.
3. Arrays can be used in speech processing where every speech signal is an array.
4. The screen of the computer is also displayed by an array. Here we use a multi-dimensional array.
5. The array is used in many management systems like a library, students, etc.
6. It is used in image processing.

➤ Types of Arrays

Arrays are classified into two types based on their dimensions:

1. One-Dimensional array

The simplest type of arrays, one-dimensional arrays, represents a linear collection of data contains a single row of elements. These arrays are usually indexed from (0 to $n-1$), where ' n ' is the size of the array. Utilizing its assigned index number, each element in an array can be conveniently accessed. (يمكن الوصول الى كل عنصر في المصفوفة باستخدام رقم الفهرس المخصص له)

Let's us look at the major types of arrays in data structures:

2. Two-Dimensional (Matrix)

Two-dimensional array represents a mathematical matrix, since they can be thought of as a grid that lays out the elements into rows and columns. Each element within the two-dimensional array can be accessed individually by its row and column location.

Declare an array التصريح عن المصفوفة

In [Python](#), the most common way to create an array-like structure is by using a list, which is dynamic and can hold mixed data types. For homogeneous data and When memory efficiency is important, the array module in the [NumPy](#) library can be used for large arrays of basic **homogeneous data types** (like integers or floats).

- To import the module:
`import numpy as np`
- To **create** integer array:
`int_array = np.array(1, 2, 3, 4, 5)`
- To **create** double (float) array:
`float_array = np.array([1.1, 2.2, 3.3, 4.4])`
- To **create** character array:
`char_array = np.array(['a', 'b', 'c', 'd'])`

➤ Access the Elements of an Array

The array element can be accessed **referring to its index number inside square brackets []**.

1. **To change the value of a 3th element in the char_array to X, and print all the element in the array.**

```
char_array[2] = X           // The outputs is
print(char_array)          [a, b, X, d]
```

2. **Using For loops to print the value of each element in Array separately:**

```
for element in char_array:  // The outputs is
    print(element)          a
                             b
                             X
                             D
```

➤ Two-Dimensional Array

A two-dimensional array is a data structure containing a set of cells arranged in as a table with rows and columns, although the values are stored **linearly** in memory. It also can call it as a matrix. It can be of any type like integer, character, float, etc. depending on the initialization. Each cell in the array can be accessed through **two** indexes that specify the **row** number (**n**) and the **column** number (**m**) respectively. The following table represent a two-dimensional array.

	Col1	Col2	Col3	Col4
--	------	------	------	------

Row1	a [0][0]	a [0][1]	a [0][2]	a [0][3]
Row2	a [1][0]	a [1][1]	a [1][2]	a [1][3]
Row3	a [2][0]	a [2][1]	a [2][2]	a [2][3]
Row4	a [3][0]	a [3][1]	a [3][2]	a [3][3]

2D Array Representation

Representation 2D array in memory

A two-dimensional array is stored in memory as contiguous storage locations. The elements of each row are stored together contiguously, and the rows are stored one after the other.

Define the 2D arrays:

- To **create** a 2D array with (4) rows and (2) column to store integer data:

```
int_array = np.array([[1234, 56],
                      [1212, 33],
                      [1434, 80],
                      [1312, 78]])
```
- To **create** a 2D array with (4) rows and (2) column filled with zeros

```
array_2d_np = np.zeros((3, 4))
```
- To **create** a 2D array with (4) rows and (2) column filled with a value 5

```
array_2d_filled = np.full((3, 4), 5)
```

Python program to Print a 2D Array

In many cases, we may need to print a resultant 2D array after performing some operations on it. So how do we do that?

There are several ways in Python to print a 2D array:

- Using simple **print()** functions

```
import numpy as np
data = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(data)
```

Output:

```
[[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

- Using Nested Loops for Formatted Output

1. The first outer (**for**) will traverse the rows from index 0 to count_of_rows -1.
 2. The second inside (**for**) will traverse the column from index 0 to count_of_coulmn -1.
- For a grid-like, "pretty" output that resembles a matrix, you can use nested (**for**) loops:

```
data = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
for row in data:
    for element in row:
        # Print each element with a space, but no newline
        print(element, end=' ')
    # Print a newline character after each row is finished
    print()
```

Output:

```
1 2 3
4 5 6
7 8 9
```

A more "Pythonic" and compact way to achieve the same formatted output uses the unpacking operator (*) and sep with join:

```
for row in data:
    print(*row)
```

Output:

```
1 2 3
4 5 6
7 8 9
```

- **Using NumPy for Array Operations**

If you are working with numerical data, the **NumPy** library is the standard for array manipulation and often provides more readable default output for large matrices.

```
import numpy as np
# Create a NumPy array
numpy_array = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(numpy_array)
```

Output:

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

Home work:

- Q1: Write a Python program to **create** one-dimensional Arrays of **10 integer** numbers.
- Q2: Write a Python program to **Calculate** and **Print** the **Sum** and the **Average** of all **elements** in the Arrays
- Q3: Write a Python program to **Display Largest Element** in the Array.